

Analyzing architectural microstructure in Transformers through homogenization

December 8, 2025

Contents

1 Preliminaries	1
1.1 Neural Networks	1
1.1.1 Feedforward Neural Network	2
1.1.2 Other NN architectures	5
1.2 Optimal Transport	6
1.3 Homogenization	7
1.3.1 Γ -convergence	7
1.3.2 Periodic Homogenization	8
1.3.3 Stochastic Homogenization	9
2 Transformers	10
2.1 ML Perspective	10
2.2 Dynamical System Perspective	11
3 Transformer Architectures	13
3.1 Rotary Position Embedding	13
3.2 Swin Transformer	13
4 Limits	14
4.1 Number of Particles	14
4.2 Inverse Temperature	14
4.3 Spatial Scaling	14

1 Preliminaries

1.1 Neural Networks

1.1 Definition [Statistical Learning Problem]

Let $(\mathcal{X}, \mathcal{Y})$ be the input and output spaces. Suppose we have a dataset of N independent

samples

$$\mathcal{D} = \left(x_i, y_i \right)_{i=1}^N \subset \mathcal{X} \times \mathcal{Y},$$

drawn from an unknown joint probability distribution $P_{X,Y}$.

We assume that there exists a true target mapping ($f^* : x \mapsto y$) that relates the dataset, up to some possible noise. So, we would like to learn and approximate it by a function parametrized by $\theta \in \Theta$

$$f_\theta : \mathcal{X} \rightarrow \mathcal{Y}$$

which minimizes the **expected loss (risk)**

$$\mathcal{R}(\theta) = \mathbb{E}_{(X,Y) \sim P_{X,Y}} [r(f_\theta(X), Y)],$$

for a fixed elementary loss (risk)

$$r : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_+$$

Since $P_{X,Y}$ is unknown, we typically minimize the **empirical risk**, which we will simply call **loss function**, over the dataset:

$$\widehat{\mathcal{R}}_{\mathcal{D}}(\theta) = \mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N r(f_\theta(x_i), y_i)$$

The goal is to find

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta)$$

And so f_{θ^*} would be our approximation of a target mapping.

1.1.1 Feedforward Neural Network

1.2 Definition [Feedforward Neural Network **FFNN**]

Fix depth L . A feedforward neural network consists of layers

$$0, 1, \dots, L, L+1,$$

where layer 0 is the input layer, layers $1, \dots, L$ are the hidden layers, and layer $L+1$ is the output layer.

For each layer $\ell = 1, \dots, L + 1$, let its dimension be n^ℓ and

$$W^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}, \quad b^{(\ell)} \in \mathbb{R}^{n_\ell},$$

be the parameters

$$\theta = \left((W^{(1)}, b^{(1)}), \dots, (W^{(L+1)}, b^{(L+1)}) \right),$$

Let $\sigma_\circ^{(\ell)} : \mathbb{R} \rightarrow \mathbb{R}$ be an activation function, and $\sigma^{(\ell)} : \mathbb{R}^{n_\ell} \rightarrow \mathbb{R}$ its vectorized version. Given an input vector $a^{(0)} \in \mathbb{R}^{n_0}$, the network computes recursively

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, \quad a^{(\ell)} = \sigma^{(\ell)}(z^{(\ell)}), \quad \ell = 1, \dots, L + 1.$$

The function realized by the network is the mapping

$$f_\theta : \mathbb{R}^{n_0} \rightarrow \mathbb{R}^{n_{L+1}}, \quad f_\theta(x) = a^{(L+1)} \text{ with } a^{(0)} = x$$

$a^{(0)}$ input vector, $f_\theta(a^{(0)}) = a^{(L+1)}$ output vector / realization,

$z^{(\ell)}$ pre-activation, $a^{(\ell)}$ activation,

$\sigma^{(\ell)}$ vectorized activation function / nonlinearity,

$L + 1$ depth, L number of hidden layers, n_ℓ number of neurons at the layer ℓ .

1.3 Example [1 hidden layer FFNN]

Let the depth be $L = 1$. Choose dimensions n_0, n_1, n_2 . The network has parameters

$$W^{(1)} \in \mathbb{R}^{n_1 \times n_0}, \quad b^{(1)} \in \mathbb{R}^{n_1},$$

$$W^{(2)} \in \mathbb{R}^{n_2 \times n_1}, \quad b^{(2)} \in \mathbb{R}^{n_2},$$

and two activation functions $\sigma^{(1)}$ and $\sigma^{(2)}$, each acting componentwise.

Given an input $x \in \mathbb{R}^{n_0}$, the computation is

$$z^{(1)} = W^{(1)}x + b^{(1)}, \quad a^{(1)} = \sigma^{(1)}(z^{(1)}),$$

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}, \quad a^{(2)} = \sigma^{(2)}(z^{(2)}).$$

The function represented by the network is therefore

$$f_{\theta}(x) = \sigma^{(2)}(W^{(2)} \sigma^{(1)}(W^{(1)}x + b^{(1)}) + b^{(2)}),$$

where $\theta = (W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)})$.

Example of choices: $n_2 = 1$, $\sigma_{\circ}^{(1)}(t) = t_+$, $\sigma_{\circ}^{(2)}(t) = (1 + e^{-t})^{-1}$

1.4 Theorem [Cybenko FFNN]

Let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be a continuous sigmoidal function. Let $f \in C([0, 1]^d)$. Then, for any $\varepsilon > 0$ there exist an integer N , real coefficients $a_i, b_i \in \mathbb{R}$, and weight vectors $w_i \in \mathbb{R}^d$ such that

$$\left| f(x) - \sum_{i=1}^N a_i \sigma(w_i \cdot x + b_i) \right| < \varepsilon \quad \forall x \in [0, 1]^d$$

In other words, 1 hidden layer FFNN of a sufficient width n_1 can approximate any continuous function.

More general versions of universal approximators theorems exist. They generalize in depth and dimensions, choices for σ , and approximations on the compact subsets of unbounded domains.

1.5 Proposition [Backpropagation]

Consider a feedforward neural network depth L . Let the loss be some regular mapping

$$\mathcal{L}(\theta) = \mathcal{L}(W^{(1)}, b^{(1)}, \dots, W^{(L+1)}, b^{(L+1)})$$

with θ denoting all parameters. Let $\sigma^{(\ell)} = \sigma$ and σ' denote the vectorized $\sigma_{\circ}$. Then,

Output-layer error. The error at layer $L + 1$ is

$$\delta^{(L+1)} = \nabla_{a^{(L+1)}} \mathcal{L} \odot \sigma'(z^{(L+1)})$$

Backward error propagation. For $\ell = L, L - 1, \dots, 1$, the errors satisfy

$$\delta^{(\ell)} = \left(W^{(\ell+1)} \right)^{\top} \delta^{(\ell+1)} \odot \sigma'(z^{(\ell)})$$

Gradients with respect to parameters. For each $\ell = 1, \dots, L + 1$,

$$\nabla_{W^{(\ell)}} \mathcal{L} = \delta^{(\ell)} (a^{(\ell-1)})^\top \quad \nabla_{b^{(\ell)}} \mathcal{L} = \delta^{(\ell)}$$

1.6 Proposition [Vanishing Gradient Problem]

Consider a feedforward neural network with hidden depth L . Let $\sigma_\circ$ be differentiable with

$$0 \leq \sigma'_\circ(x) \leq c < 1 \quad \text{for all } x \in \mathbb{R}.$$

Then for any $1 \leq \ell \leq L$,

$$\|\delta^{(\ell)}\| \leq c^{L+1-\ell} \left\| W^{(\ell+1)} \right\| \dots \left\| W^{(L+1)} \right\| \|\delta^{(L+1)}\|$$

In particular, if

$$c \|W^{(j)}\| < 1 \quad \forall j = \ell + 1, \dots, L + 1,$$

then

$$\|\delta^{(\ell)}\| \longrightarrow 0 \quad \text{as } L - \ell \rightarrow \infty,$$

so the gradients with respect to parameters in early layers vanish exponentially with depth.

1.1.2 Other NN architectures

1.7 Definition [Convolutional Neural Network **CNN**]

1.8 Definition [Recurrent Neural Network **RNN**]

1.9 Definition [Residual Neural Network **ResNet**]

Fix depth $L + 1$, and for each layer $\ell = 1, \dots, L + 1$ denote its dimension n^ℓ and an activation function $\sigma_\circ^{(\ell)}$. Let

$$W^{(\ell)} \in \mathbb{R}^{n_\ell \times n_{\ell-1}}, \quad b^{(\ell)} \in \mathbb{R}^{n_\ell},$$

be the parameters

$$\theta = \left((W_1^{(1)}, b_1^{(1)}, W_2^{(1)}, b_2^{(1)}), \dots, (W_1^{(L+1)}, b_1^{(L+1)}, W_2^{(L+1)}, b_2^{(L+1)}) \right),$$

Given an input vector $a^{(0)} \in \mathbb{R}^{n_0}$, the network computes recursively

$$\begin{aligned} z^{(\ell)} &= W_1^{(\ell)} a^{(\ell-1)} + b_1^{(\ell)} \\ a^{(\ell)} &= a^{(\ell-1)} + W_2^{(\ell)} \sigma^{(\ell)}(z^{(\ell)}) + b_2^{(\ell)} \end{aligned}$$

The function realized by the network is the mapping

$$f_\theta : \mathbb{R}^{n_0} \rightarrow \mathbb{R}^{n_{L+1}}, \quad f_\theta(x) = a^{(L+1)} \text{ with } a^{(0)} = x$$

1.10 Definition [Neural ODE for ResNet]

Let $\sigma_\circ$ be an activation function. Let

$$\begin{aligned} F : \mathbb{R}^d \times U &\rightarrow \mathbb{R}^d \\ (x, u) &\mapsto W_2 \sigma(W_1 x + b_1) + b_2 \end{aligned}$$

with control

$$u(t) = (W_1(t), b_1(t), W_2(t), b_2(t)) \in L^\infty([0, 1]; U)$$

A neural ODE is then a controlled dynamical system

$$\begin{cases} \dot{x}(t) = F(x(t), u(t)) \\ x(0) = x_0 \end{cases}$$

So that, the system is driven from an input vector $x(0) \in \mathbb{R}^d$ to a prediction output $x_u(1) \in \mathbb{R}^d$ by u . The training procedure is then the choice of a suitable control u .

1.2 Optimal Transport

1.11 Definition [Monge Problem]

1.12 Definition [Kantorovich Problem]

1.13 Definition [Wasserstein Distances]

1.3 Homogenization

1.3.1 Γ -convergence

1.14 Definition [Γ -convergence]

Let (X, d) be a metric space and let $\varepsilon > 0$ index a family of functionals

$$\mathcal{J}_\varepsilon : X \rightarrow \mathbb{R}$$

with a limiting functional

$$\mathcal{J}_0 : X \rightarrow \mathbb{R}$$

and the corresponding minimization problems

$$(\mathcal{P}_\varepsilon) : \min_{u \in X} \mathcal{J}_\varepsilon(u) \quad (\mathcal{P}_0) : \min_{u \in X} \mathcal{J}_0(u)$$

We define a notion of convergence for minimization problems in the sense that

$$(\mathcal{P}_\varepsilon) \xrightarrow[\varepsilon \rightarrow 0]{\Gamma(d)} (\mathcal{P}_0)$$

iff for any $u_0 \in X$ we have

1) $\Gamma - \liminf$, for any sequence $(u_\varepsilon)_\varepsilon$ that converges to u_0 in d , one has

$$\liminf_{\varepsilon \rightarrow 0} \mathcal{J}_\varepsilon(u_\varepsilon) \geq \mathcal{J}_0(u_0)$$

2) $\Gamma - \limsup$, there exists a sequence $(u_\varepsilon)_\varepsilon$ that converges to u_0 in d , such that

$$\limsup_{\varepsilon \rightarrow 0} \mathcal{J}_\varepsilon(u_\varepsilon) \leq \mathcal{J}_0(u_0)$$

The relaxed version of this definition requires that for any $\eta > 0$, there exists a sequence

$(u_\varepsilon)_\varepsilon$ that converges to u_0 in $\mathbf{d}$, such that

$$\limsup_{\varepsilon \rightarrow 0} \mathcal{J}_\varepsilon(u_\varepsilon) \leq \mathcal{J}_0(u_0) + \eta$$

There are even more relaxed versions of this definition, such that the following theorem is still true.

1.15 Theorem

Let $(X, \mathbf{d})$ be a metric space, let

$$(\mathcal{P}_\varepsilon) \xrightarrow[\varepsilon \rightarrow 0]{\Gamma(\mathbf{d})} (\mathcal{P}_0)$$

and $(u_\varepsilon)_\varepsilon$ be a family of solutions of $(\mathcal{P}_\varepsilon)_{\varepsilon > 0}$ such that

$$u_\varepsilon \xrightarrow[\varepsilon \rightarrow 0]{\mathbf{d}} u_0$$

Then, u_0 solves $(\mathcal{P}_0)$.

1.3.2 Periodic Homogenization

1.16 Lemma

Let $b \in L^\infty(\mathbb{R})$ with period 1 and $\phi \in L^1(\mathbb{R})$. Then,

$$\lim_{\varepsilon \rightarrow 0} \int_{\mathbb{R}} b\left(\frac{x}{\varepsilon}\right) \phi(x) \, dx = \langle b \rangle \int_{\mathbb{R}} \phi(x) \, dx$$

1.17 Example [1-D toy case]

$$\left| \begin{array}{l} - \left[a \left(\frac{x}{\varepsilon} \right) u'_\varepsilon(x) \right]' = f(x), \quad x \in (0, 1), \\ u_\varepsilon(0) = u_\varepsilon(1) = 0 \end{array} \right.$$

As $\varepsilon \rightarrow 0$, it goes to an effective equation

$$\left| \begin{array}{l} - [a_\star u'_\star(x)]' = f(x), \quad x \in (0, 1), \\ u_\star(0) = u_\star(1) = 0 \end{array} \right.$$

with

$$a_\star = \left\langle \frac{1}{a} \right\rangle^{-1} = \left(\int_0^1 \frac{1}{a(x)} dx \right)^{-1}$$

Ansatz

Assume the multiscale expansion of the form

$$u_\varepsilon(x) = u_0(x, x/\varepsilon) + \varepsilon u_1(x, x/\varepsilon) + \varepsilon^2 u_2(x, x/\varepsilon) + \dots$$

After matching coefficients, heuristically gives

$$-\operatorname{div}_x(A^\star \nabla_x u_0) = f(x)$$

with

$$A_{j,k}^\star = \int_Y \left(A_{j,k}(y) + \sum_{l=1}^d A_{j,l} \partial_{y_l} \omega_k(y) \right) dy$$

where ω_k is a corrector, solving

$$-\operatorname{div}_y(A(y) \nabla_y \omega_k)(y) = \operatorname{div}_y(A(y) e_k)$$

Two-scale convergence

1.18 Definition [Two-scale convergence]

1.19 Theorem

1.3.3 Stochastic Homogenization

[Gloria-Otto], [Armstrong-Kuusi-Mourrat]

2 Transformers

[Vas+23]

2.1 ML Perspective

2.1 Definition [Self-Attention Mechanism]

Let $(x_1, \dots, x_n)$ be a list of vectors with $x_i \in \mathbb{R}^d$. Let

$$Q, K, V \in \mathbb{R}^{d \times d}$$

be a triple of linear maps.

Fix a token i and for $j \in [n]$, set the **query** of i , the **key** of j , and the **value** of j

$$q_i = Qx_i, \quad k_j = Kx_j, \quad v_j = Vx_j$$

For a fixed $\beta > 0$, define the similarity kernel

$$w_{ij} = e^{\beta s_{ij}} = e^{\beta \langle q_i | k_j \rangle}$$

which, after normalization by the partition function,

$$Z_{\beta,i} = \sum_{k=1}^n e^{\beta s_{ik}}$$

give the **attention weight** of i on j

$$A_{ij} = \frac{w_{ij}}{Z_i}$$

Finally, compute the new representation token (**context vector**) of i

$$y_i = \sum_{j=1}^n A_{ij} v_j$$

Repeat for all $i \in [n]$.

2.2 Definition [Multi-Head Attention]

Let $(x_1, \dots, x_n)$ be a list of vectors with $x_i \in \mathbb{R}^d$. Let $H \leq d$ and

$$Q_h, K_h, V_h \in \mathbb{R}^{d_h \times d}$$

be a triple of matrices for $h \in [H]$ and $\sum_h d_h = d$.

For each h , compute as before

$$y_i^{(h)} = \sum_{j=1}^n A_{ij}^{(h)} v_j^{(h)} \in \mathbb{R}^{d_h}$$

and concatenate

$$y_i = (y_i^{(1)}, \dots, y_i^{(H)}) \in \mathbb{R}^d$$

Then, for some $O \in \mathbf{SO}(d)$, compute

$$u_j = O(y_j)$$

2.3 Definition [Positional Encoding]

Attention mechanism applied to a prompt $(x_i)_{i \in [n]}$ is permutation invariant. For this, each token is represented as

$$x_i = w_i + p_i$$

Where w is a word embedding and p is a positional encoding that breaks the invariance.

2.4 Definition [Transformer (ML)]

2.2 Dynamical System Perspective

[Ges+25]

2.5 Definition [Transformer (Neural ODE)]

Take a transformer with residual connection. The refined discretization of

$$x^{(\ell+1)} = x^{(\ell)} + y^{(\ell)}$$

will give the dynamics $\dot{x}$, while the RMS normalization will introduce the projector

onto a tangent spaces to a unit sphere

Let $Q, K, V = I_d$

$$\left| \begin{array}{l} \dot{x}_i(t) = \mathbf{P}_{x_i(t)}^\perp \left(Z_{\beta,i}^{-1} \sum_{j=1}^n e^{\beta \langle x_i(t) | x_j(t) \rangle} x_j(t) \right) \\ x_i(0) \in \mathbb{S}^{d-1} \end{array} \right. \quad (\text{SA})$$

For general Q, K, V

$$\left| \begin{array}{l} \dot{x}_i(t) = \mathbf{P}_{x_i(t)}^\perp \left(\sum_{j=1}^n A_{ij}(t) V(t) x_j(t) \right) \\ x_i(0) \in \mathbb{S}^{d-1} \\ A_{ij} = Z_{\beta,i}^{-1} \sum_{k=1}^n \exp(\beta \langle Q(t) x_i(t) | K(t) x_k(t) \rangle) \end{array} \right. \quad (\text{SA}+)$$

Surrogate model

$$\left| \begin{array}{l} \dot{x}_i(t) = f_i(x) = \mathbf{P}_{x_i(t)}^\perp \left(\frac{1}{n} \sum_{j=1}^n e^{\beta \langle x_i(t) | x_j(t) \rangle} x_j(t) \right) \\ x_i(0) \in \mathbb{S}^{d-1} \end{array} \right. \quad (\text{USA})$$

2.6 Definition [Transofmer (Measure-Flow)]

$$\dot{x}_i(t) = \mathcal{V}[\mu(t)](x_i(t))$$

$$\mu(t) = \frac{1}{n} \sum_{i \in [n]} \delta_{x_i(t)}$$

$$Z_{\beta, \mu(t)}(x) = \int e^{\beta \langle x | y \rangle} d\mu(y)$$

$$\mathcal{V}[\mu] : \mathbb{S}^{d-1} \rightarrow \mathbb{T}\mathbb{S}^{d-1}$$

$$x \mapsto \mathbf{P}_x \left(Z_{\beta, \mu}^{-1}(x) \int e^{\beta \langle x | y \rangle} y d\mu(y) \right)$$

$$\left\{ \begin{array}{ll} \partial_t \mu + \operatorname{div}(\mathcal{V}[\mu]\mu) = 0, & \mathbb{R}_+ \times \mathbb{S}^{d-1} \\ \mu|_{t=0} = \mu(0), & \mathbb{S}^{d-1} \end{array} \right. \quad (\text{Continuity})$$

2.7 Definition [Interaction Energy]

$$E_\beta[\mu] = \frac{1}{2\beta} \iint e^{\beta \langle x | x' \rangle} \mathrm{d}\mu(x) \mathrm{d}\mu(x')$$

3 Transformer Architectures

3.1 Rotary Position Embedding

[Su+23] Instead of

$$x = w + p$$

representation, use rotation

$$x = Rw$$

3.2 Swin Transformer

[Liu+21]

4 Limits

4.1 Number of Particles

$$n \rightarrow \infty$$

4.2 Inverse Temperature

$$\beta \rightarrow \infty$$

4.3 Spatial Scaling

$$\varepsilon \rightarrow 0$$

References

- [Ges+25] Borjan Geshkovski, Cyril Letrouit, Yury Polyanskiy, and Philippe Rigollet. *A mathematical perspective on Transformers*. 2025. arXiv: [2312.10794 \[cs.LG\]](#). URL: <https://arxiv.org/abs/2312.10794>.
- [Su+23] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. 2023. arXiv: [2104.09864 \[cs.CL\]](#). URL: <https://arxiv.org/abs/2104.09864>.
- [Vas+23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. *Attention Is All You Need*. 2023. arXiv: [1706.03762 \[cs.CL\]](#). URL: <https://arxiv.org/abs/1706.03762>.
- [Liu+21] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. *Swin Transformer: Hierarchical Vision Transformer using Shifted Windows*. 2021. arXiv: [2103.14030 \[cs.CV\]](#). URL: <https://arxiv.org/abs/2103.14030>.